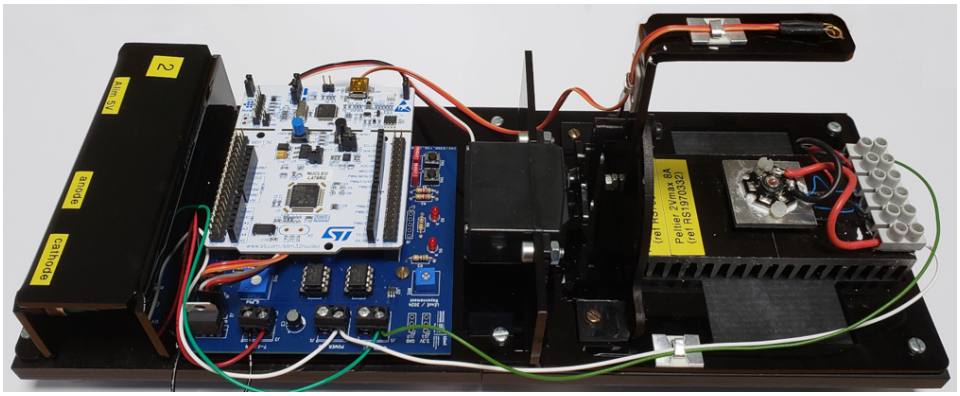
Institut d'Optique Graduate School
INTERFAÇAGE NUMÉRIQUE

Travaux Pratiques

Semestre 6

Banc de mesure de rayonnement

2 séances



Ce sujet est disponible au format électronique sur le site du LEnsE - https ://lense.institutoptique.fr/ dans la rubrique Année / Première Année / Interfaçage Numérique S6 / Bloc 3 – Interface d'acquisition de données.

© 2026 by LEnsE-IOGS

Banc de mesure de rayonnement

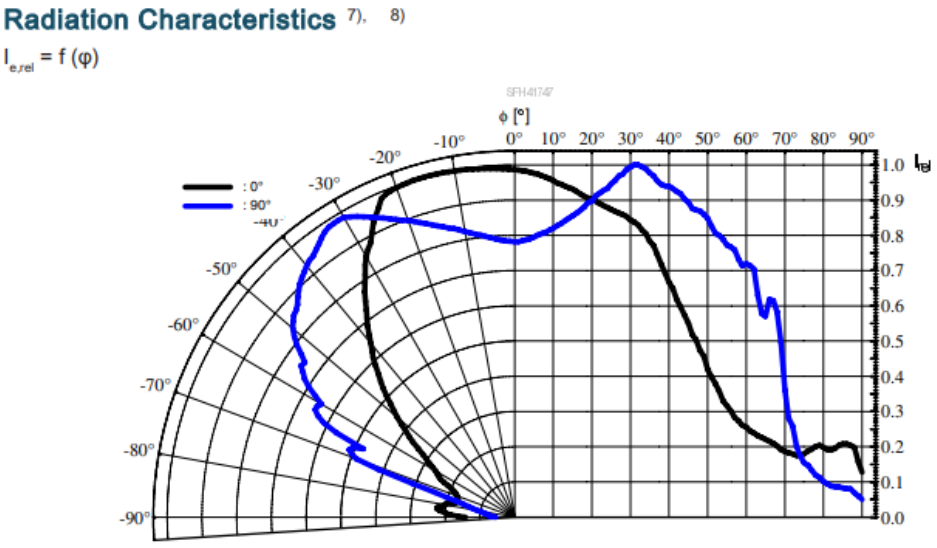
À l'issue des séances de TP concernant le **bloc d'acquisition d'un diagramme de rayonnement**, les étudiant·es seront capables de :

- Développer, mettre en œuvre et valider une **solution informatique pour acquérir des données** selon un protocole établi
- Mettre en œuvre un **protocole simple de communication** entre un ordinateur et un microcontrôleur pour transmettre des commandes et lire des données
- Documenter un programme informatique
- Etablir un **protocole de test**, réaliser les tests et rédiger une **analyse préliminaire des résultats**

Objectifs du mini-projet

L'objectif principal de ce mini-projet est d'**automatiser un banc de mesure de rayonnement lumineux** à l'aide d'un ordinateur et d'une interface en Python. Le matériel est piloté par une carte à microcontroleur à laquelle il faudra envoyer des commandes selon un protocole série donné.

Un diagramme de rayonnement lumineux est la **représentation graphique** de la **distribution angulaire** d'une grandeur caractérisant le rayonnement d'une source lumineuse.



Exemple de diagramme de rayonnement d'une LED OSRAM SFH 41747 (documentation technique)

Vous aurez à votre disposition une **maquette** permettant la mise en mouvement d'un système de photodétection autour d'une source de puissance à LED. Cette maquette est pilotée par une carte Nucléo (contenant un microcontroleur).

Déroulement du bloc

Séance 1 / Prise en main de la maquette et du protocole

Etape A Etude de la maquette et du protocole

Etape B Liaison série et python

Séance 2 / Mise en place de l'application d'acquisition

Etape C Application d'acquisition en Python

Etape D Essai sur une LED de puissance

Selon le temps restant, il sera possible d'intégrer les fonctions écrites en Python dans une interface graphique, dont la structure de base est fournie.

DISC / Accumulation de preuves

Nous vous rappelons également que l'un des livrables final de ce module est un **Document Individuel de Suivi de Compétences** qui devra revendiquer votre progression dans l'acquisition des compétences suivantes :

— C4 – Valider une solution technologique
— C7 – Travailler en équipe

Travail à réaliser

Vous devrez donc tout au long des séances de TP :

— Documenter les tests réalisés pour valider les fonctionnalités mise en oeuvre
— Stocker les preuves pour revendiquer la montée en compétences (résultats, photos, vidéos...)

Ressources

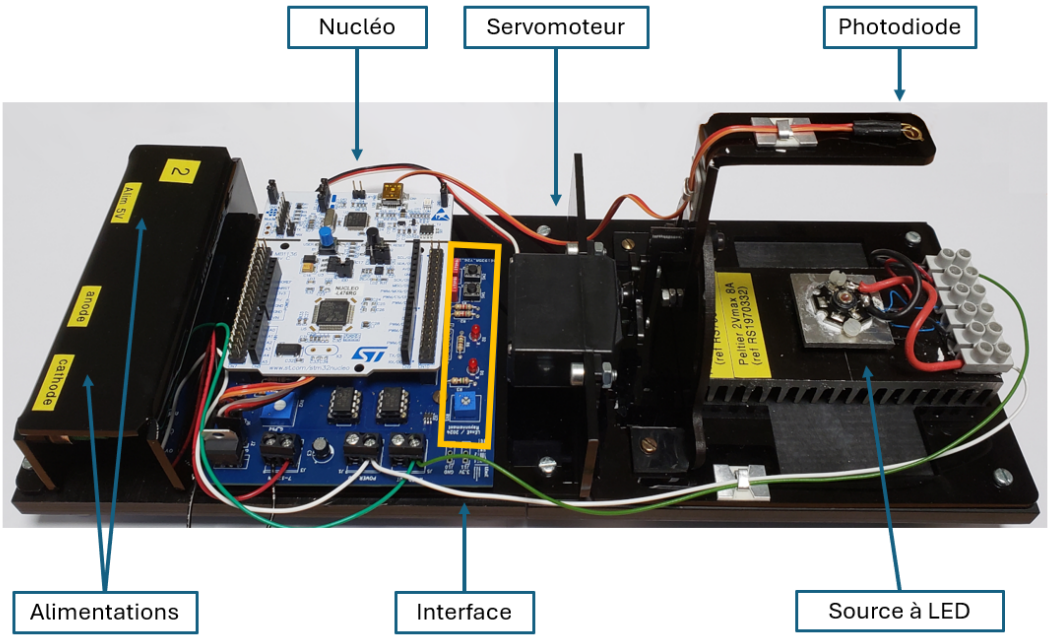
Des fichiers binaires, des codes en C++ et des scripts en Python sont à télécharger tout au long des séances de TP. Vous les trouverez sur le site du LEnsE :

Année / Première Année / Interfaçage Numérique S6 / Bloc 3 – Interface d'acquisition de données / Exemples de codes

Séance 1 / Prise en main de la maquette et du protocole

Description de la maquette

La maquette est constituée :
— d'une source de lumière, une LED de puissance
— d'une photodiode BPX65, associée à un circuit de photodétection simple
— d'un servomoteur, permettant de positionner le bras supportant la photodiode au dessus de la source lumineuse selon un angle donné (de -90° à +90°, par pas minimal de 1°)
— d'une carte de contrôle associée à une carte Nucléo L476RG



Alimentation électrique

Les bornes d'alimentation notées **Alim** 7 V permet d'alimenter le servomoteur.
Les bornes notées **anode** et **cathode** permettent d'alimenter la LED de puissance.

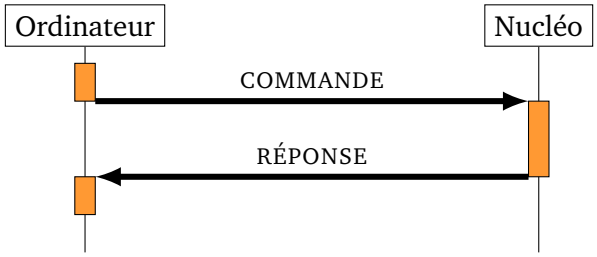
La tension maximale admissible par le servomoteur est de 7 V !
La tension minimale est de 5.5 V

Le courant maximal admissible par la LED de puissance est de 200 mA !

Protocole de communication

Un **protocole de communication** est un ensemble de règles, de conventions et de formats définis pour permettre l'échange d'informations entre différents systèmes. Il détermine comment les données sont structurées, envoyées, reçues et interprétées par les parties communicantes.

Dans le cas de cette application, l'ordinateur, maitre de la communication, devra envoyer des commandes à la carte embarquée, qui devra les exécuter puis acquitter du bon déroulement de l'opération.



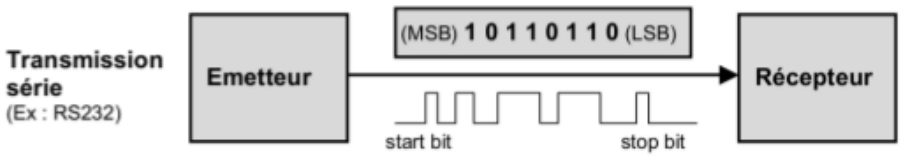
Il est important lorsqu'on souhaite **construire un protocole de communication** de :

— **choisir un protocole de bas niveau** pour la transmission des commandes et des réponses
— définir la **structure des commandes**
— planifier la **liste des commandes**
— coder la récupération des données sur la carte Nucléo (et tester avec un moniteur Série)
— coder l'envoi des commandes sur le PC (en Python par exemple)

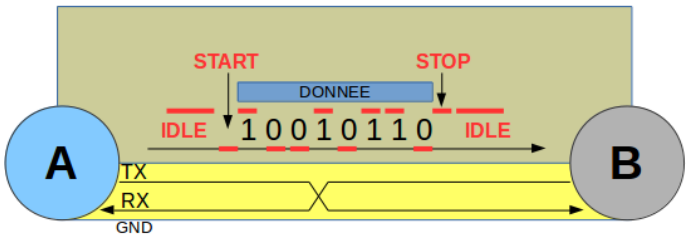
Choix du protocole de bas niveau

Nous utiliserons dans le cas de notre application la **liaison Série** (ou RS232).

Le protocole RS-232 est une **norme de communication série asynchrone** point à point qui permet de relier un équipement terminal de traitement de données (comme un ordinateur) à un équipement de terminaison de circuit de données (comme un instrument de mesure).



Il permet la communication entre deux noeuds d'un réseau, selon une trame structurée de la façon suivante : 1 bit de *start*, 8 bits de données (en règle générale), 1 bit de parité optionnel (pair ou impair), 1 à 2 bits de stop.



*Nous utiliserons le logiciel **Tera Term** afin de pouvoir visualiser et transmettre des données au format RS-232 à la maquette.*

Etape A / Etude de la maquette et du protocole

➜ **M** Connecter la maquette à l'aide du câble USB.

Elle est par la suite reconnue comme un périphérique de stockage USB, dans lequel vous pourrez ***déplacer les fichiers binaires*** *à télécharger tout au long des différentes étapes.*

A.1 / Etude du protocole de bas niveau

Le protocole choisi est RS-232 (voir page précédente). Il permet d'envoyer un caractère ASCII à la fois.

L'exemple, que nous allons étudier dans un premier temps, se base sur un protocole simplifié, ne nécessitant l'envoi que d'**un seul caractère par commande**.

➜ **M** Télécharger le fichier *test_comm_led.bin* (dans les ressources du TP sur le site du LEnsE). Copier-le dans sur la carte Nucléo de la maquette.

➜ **M** Lancer le logiciel **Tera Term**. Sélectionner, dans la partie **Série**, le port de communication correspondant à la carte Nucléo (STM32...).

➜ **M** Sur le logiciel **Tera Term** :
— cocher l'option *Echo Local* dans le menu *Configuration / Terminal*
— modifier la vitesse de transfert à **115200 bauds** dans le menu *Configuration / Port Série...*

➜ **M** Dans **Tera Term**, envoyer les caractères 'a', 'e' et 'z' à la maquette. Observer ce qui se passe sur les différentes LEDs (D1 et D2).

➜ **M** Changer la vitesse de transfert à 9600 bauds et tester à nouveau l'envoi des commandes. Que se passe-t-il ?

Le code embarqué dans la maquette est fourni dans le fichier *test_comm_led.cpp* (dans les ressources du TP sur le site du LEnsE).

➜ **M** Télécharger ce fichier et l'ouvrir avec un éditeur de texte.

➜ **Q** Faire un schéma de principe du fonctionnement de ce code et des échanges entre le PC et la maquette.

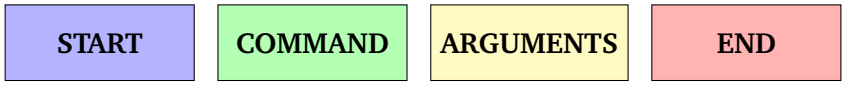
A.2 / Protocole complet

Le protocole de bas niveau ne suffit pas lorsqu'on souhaite envoyer des commandes (ou des données) plus complexes qu'un simple caractère.
Il est alors essentiel de mettre en place un **protocole de plus haut niveau**.

La ***liste des commandes*** *utiles pour la maquette sont disponibles en annexe de ce document.*

Structure des commandes

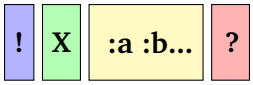
Chaque commande doit avoir un **format bien défini** pour que l'instrument piloté puisse la comprendre et la traiter.



La liaison Série étant asynchrone, il est indispensable de prévoir dans le protocole des **caractères de début et de fin** de chaines de commande et de réponse.

Commande

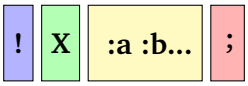
Par exemple, on pourra se baser sur la structure suivante pour le cas d'une commande (entre le PC et le microcontroleur) :



Les arguments sont facultatifs, selon les commandes.

Réponse

Par exemple, on pourra se baser sur la structure suivante pour le cas d'une réponse (entre le PC et le microcontroleur) :



Les arguments sont facultatifs, selon les commandes.

Etape B / Liaison série et python

B.1 / Premiers échanges - protocole de bas niveau

On propose de remplacer la console Série par un **script Python** contenu dans le fichier *test_comm_led.py*.

➜ **M** Exécuter ce script python.

Transmission des données par liaison série

Séance 3 / Mise en place d'un protocole de communication

Pilotage de la LED de puissance

➜ **M** Alimenter la LED de puissance avec un courant maximal de 200mA.

Attention ! Paramètrer l'alimentation de la LED pour ne pas dépasser 200mA !
Attention également à vos yeux...
Et brancher la LED dans le bon sens... Cathode = point de plus faible potentiel.

Premier diagramme de rayonnement

➜ **M** Faire l'acquisition des valeurs d'intensité lumineuse pour des angles allant de 0 à $180°$ par pas de $10°$ et transmettre les données par liaison Série à l'ordinateur.

Etape 11 / Mettre en place un protocole de communication basé sur une liste de commandes

Envoi et réception de messages simples

Avant de se lancer dans la mise en place du protocole complet, nous vous proposons de tester l'exemple donné dans le fichier *test_comm_led.cpp*. Cet exemple se base sur un protocole simplifié, ne nécessitant l'envoi que d'**un seul caractère par commande**.

➜ **M** Tester le code fourni en envoyant des lettres par l'intermédiaire d'une console Série (logiciel **TeraTerm** par exemple).

➜ **Q** A quel moment est appelée la fonction *ISR_my_pc_reception()* ? A quoi sert-elle ?

On propose de remplacer la console Série par un **script Python** contenu dans le fichier *test_comm_led.py*.

➜ **M** Exécuter ce script python.

Structure de base

On se propose de tester le code *test_version_a_d.cpp*. Ce fichier est disponible sur le site du LEnsE dans la rubrique *Année / Première Année / Interfaçage Numérique S6 / Banc de mesure de rayonnement lumineux / Exemples de Codes*.

➜ **M** Compiler ce code et le téléverser sur la carte Nucléo.

➜ **Q** A quoi sert la fonction *is_angle_ok()* ?

➜ **M** Ouvrir le logiciel TeraTerm et envoyer la commande suivante : !A :10 ?.

➜ **Q** Que renvoie la carte Nucléo ? Expliquer les fonctions *ISR_my_pc_reception()* (vis-à-vis du choix de la structure d'une commande) et la boucle infinie. On s'intéressera notamment aux changements de valeur des variables *string_complete* et *input_cnt*.

➜ **Q** Que se passe-t-il lorsqu'on transmet une mauvaise valeur (soit avec la commande A, soit avec la commande D) ?

➜ **Q** Faire un algorigramme du code.

Ajout des autres commandes

On souhaite à présent compléter ce code pour pouvoir traiter l'ensemble des commandes mentionnées dans la section précédente (T, A, B, C, S, E, D).

➜ **M** Compléter le code précédent et le tester.

➜ **Q** A quel moment doit-on utiliser la **fonction d'acquisition** réalisée à l'étape 8 ? D'autres commandes sont-elles acceptables lors de l'acquisition ? Comment bloquer l'exécution des autres commandes ?

➜ **M** Mettre en place ce blocage.

➜ **Q** A quel moment doit-on utiliser la **fonction de transmission** des données réalisée à l'étape 9 ?

Test complet

➜ **Q** Quelle est la suite de commandes à transmettre à la carte Nucléo pour permettre l'acquisition de données pour des angles allant de $-40°$ à $+60°$ par pas de $10°$ et la récupération des données sur le moniteur Série ?

➜ **Q** Tracer le diagramme de séquence associé à ces échanges de commandes et de réponses entre le PC et la carte Nucléo.

➜ **M** Tester l'acquisition de cette série de données.

Etape 12 / Utiliser la bibliothèque pySerial en Python pour envoyer les commandes à la carte Nucléo

> **ATTENTION** / Lors de l'utilisation d'un script Python nécessitant l'accès à la liaison Série, le logiciel **TeraTerm** doit être **fermé**.

On se propose à présent d'utiliser un script en langage Python pour pouvoir transmettre les commandes et recevoir les données sur l'ordinateur. Nous utiliserons la bibliothèques **pySerial** (à installer par la commande *pip install pyserial* dans une console ou un prompt d'Anaconda).

Un code de base, nommé *test_version_a_d.py*, est disponible sur le site du LEnsE dans la rubrique *Année / Première Année / Interfaçage Numérique S6 / Banc de mesure de rayonnement lumineux / Exemples de Codes*.

➜ **M** Exécuter ce code et tester les deux commandes pré-configurées ('a' et 'd').

➜ **M** A partir de cet exemple, réaliser un script qui permette de lancer l'acquisition de données pour des angles allant de $-40°$ à $+60°$ par pas de $10°$ et la récupération des données acquises dans une liste en Python.

➜ **M** Afficher les données obtenues sur un graphique à l'aide de Matplotlib. Générer l'axe des angles.

Etape 13 / Tester la communication entre la carte Nucléo et le script Python pour afficher les données

On souhaite enfin créer une application en console complète permettant à l'utilisateur de saisir les angles de départ et de fin, le pas angulaire et de lancer l'acquisition. Votre application devra ensuite afficher les données acquises.

Vous pourrez faire plusieurs essais avec des courants dans la LED de puissance de $50\,\mathrm{mA}$, $100\,\mathrm{mA}$ et $200\,\mathrm{mA}$ par exemple.

INTERFAÇAGE NUMÉRIQUE

Travaux Pratiques

Semestre 6

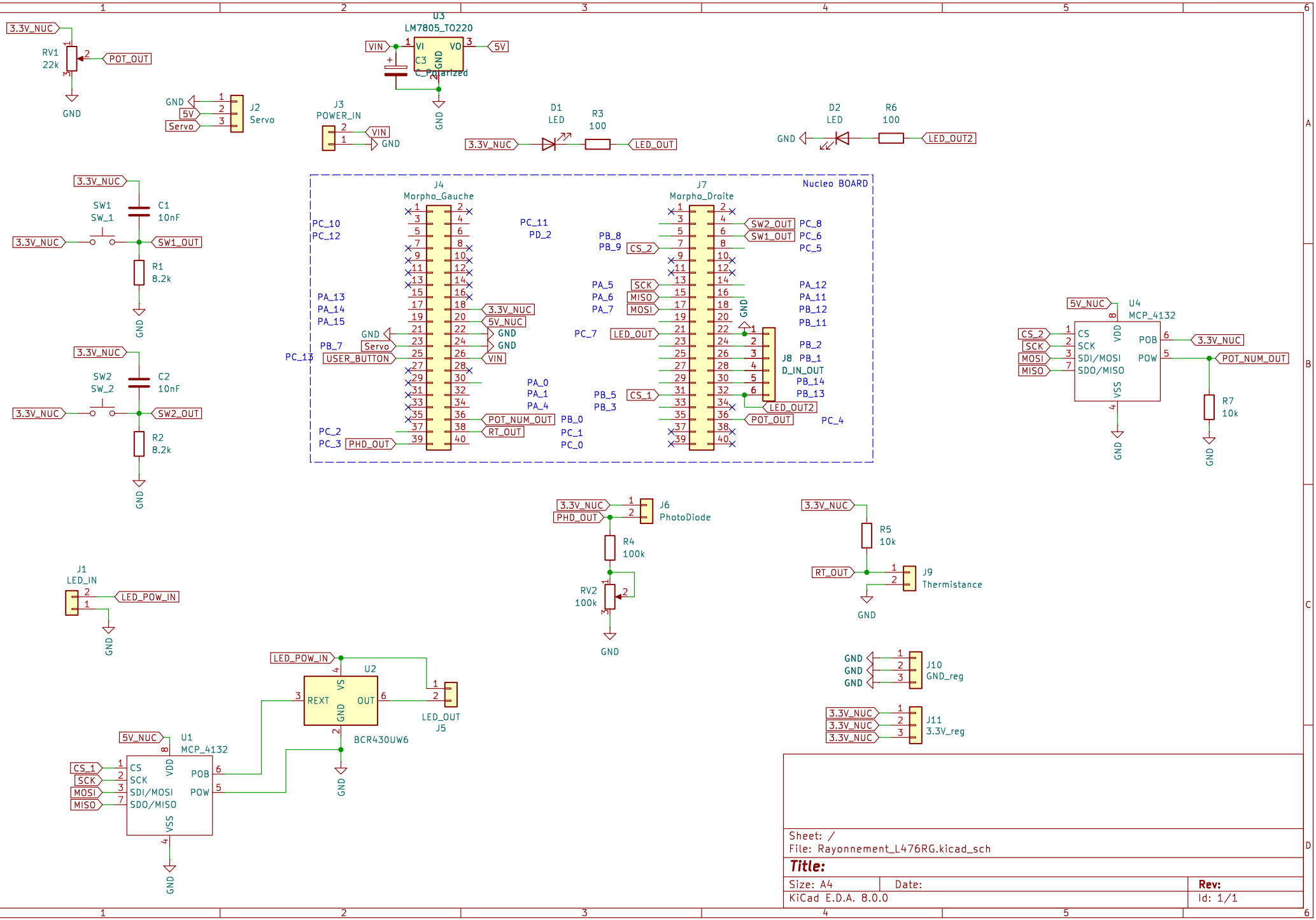
Ressources - Bloc Rayonnement

Liste des ressources

— Liste des commandes du protocole
— Schéma de la carte de la maquette de rayonnement
— PCB de la carte de la maquette de rayonnement

Liste des commandes

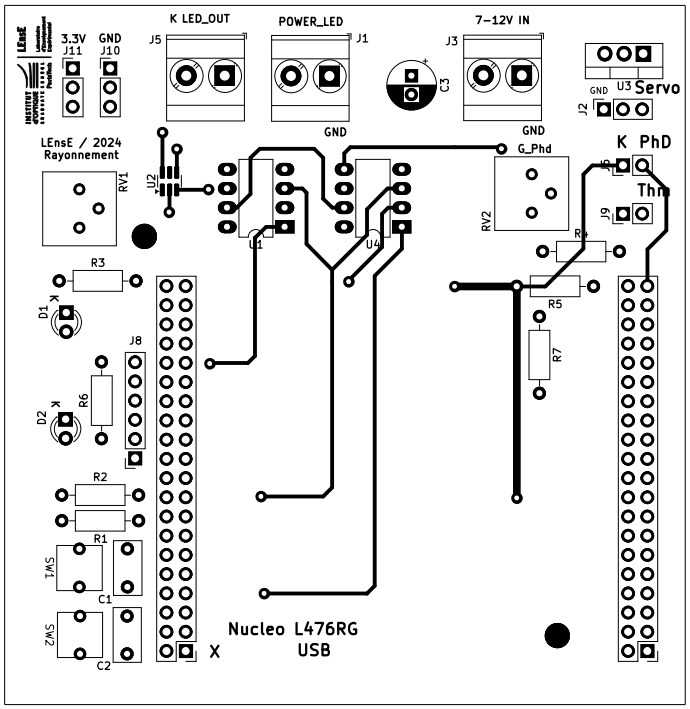
Commande PC	Description	**Réponse Nucléo**
!T ?	Test de la transmission et positionnement à 0° réglage possible avec le potentiomètre RV1	!T ;
!P ?	Validation du positionnement à 0°	!P ;
!A :VALUE ?	Transmission de l'angle de départ souhaité pour le diagramme valeur entière en degré *Si angle non compris entre -90 et 90*	!A :VALUE ; VALUE= -100
!B :VALUE ?	Transmission de l'angle final souhaité pour le diagramme valeur entière en degré *Si angle non compris entre -90 et 90*	!B :VALUE ; VALUE= -100
!C :VALUE ?	Transmission du pas angulaire souhaité pour le diagramme valeur entière en degré *Si angle non compris entre -90 et 90*	!C :VALUE ; VALUE= -100
!S ?	Lancement de l'acquisition nb est le nombre d'échantillons qui seront acquis par la carte *Si les angles fournis sont non compatibles*	!S :NB ; NB= 0
!E ?	Test de fin de l'acquisition (Y)es or (N)o	!E :Y/N ;
!D :INDEX ?	Demande de récupération d'une donnée index correspond au numéro de l'échantillon souhaité value correspond à la valeur de l'échantillon souhaité	!D :INDEX :VALUE ;
!Z :ANGLE ?	Demande de récupération d'une donnée angle correspond à l'angle voulu pour la mesure value correspond à la valeur de l'échantillon souhaité	!Z :ANGLE :VALUE ;



Sheet: /
File: Rayonnement_L476RG.kicad_sch

Title:

Size: A4	Date:	**Rev:**
KiCad E.D.A. 8.0.0		Id: 1/1


K LED_OUT
POWER_LED
7-12V IN
3.3V
GND
J11
J10
J5
J1
J3
C3
U3
Servo
GND
J2
GND
GND
LEnsE / 2024
Rayonnement
RV1
U2
U1
U4
G_Phd
RV2
K PhD
Thr
J9
R4
R3
R5
R7
D1
J8
D2
R6
R2
R1
SW1
C1
SW2
C2
Nucleo L476RG
X
USB